The DOM, the accessibility tree, and what gets kept

Week 2, Lecture 1 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- Why raw HTML cannot go into a prompt
- The DOM tree: nodes, elements, layout
- The accessibility tree: roles, names, ARIA
- Detecting interactive elements: tags, roles, and click handlers
- Visibility filtering: what gets kept, what gets dropped

Part 1: the problem with raw HTML

Raw HTML is too big and too noisy

- A typical e-commerce page: 50,000 to 500,000 characters of HTML
- LLM context windows hold tens of thousands of tokens
- Most of that HTML is structural scaffolding: nested divs, inline styles, comment nodes
- The useful information is a small fraction of the total
- Sending raw HTML wastes tokens and buries the signal

What the agent actually needs

- Which elements can I act on?
- What does each element mean (its label and role)?
- Is it currently visible in the browser?
- What integer refers to it so I can say “click 5”?

The DOM service answers all four questions and discards everything else.

Part 2: the DOM tree

The DOM tree: a quick map

- The browser parses HTML and builds a tree of nodes
- Each node: a tag name, attributes, child nodes, computed layout (bounding box)
- Root is `document`; children: `html`, `head`, `body`, then everything visible
- The bounding box tells us where an element sits on screen

What the DOM tree gives us

- Element identity: what tag is it?
- Attributes: `href`, `type`, `aria-label`, `class`
- Layout: where is it? How big is it? Is it on screen?
- Parent-child relationships: nested structure

What the DOM tree does not give us

- Semantic meaning for non-semantic elements
- A `div` with a click handler looks like any other `div`
- Class names like `btn-primary` are human conventions, not machine-readable roles
- For semantic meaning, we need the accessibility tree

Part 3: the accessibility tree

What the accessibility tree is

- A parallel structure the browser maintains alongside the DOM
- Originally built for screen readers and assistive technology
- Every node has a **role** (button, link, textbox, heading)
- Every node has a **name** (the text a screen reader would announce)
- Name sources: text content, `aria-label`, `title`, linked `<label>` element

How roles and names work

- `<button>Search</button>` → role: button, name: Search
- `<a href="/about">About us</a>` → role: link, name: About us
- `<div role="button" aria-label="Close dialog">X</div>` → role: button, name: Close dialog
- `<input type="text" placeholder="Email">` → role: textbox, name: (placeholder or linked label)

Why this matters for agents

- The agent sees “button: Search” not “div class=btn-cta-primary”
- Semantic descriptions transfer across site redesigns (class names change; roles rarely do)
- A 10x token saving: accessibility-tree text vs. screenshot (Sokolov, 2025)
- Text-only models can act on the accessibility tree; they cannot act on a screenshot

Part 4: CDP and the dual extraction

Chrome DevTools Protocol (CDP)

- A wire protocol originally built for browser DevTools
- Any client can send commands to a running Chrome/Chromium instance
- browser-use uses it directly for DOM intelligence
- Four parallel CDP calls per perceive step:
  - `DOMSnapshot.captureSnapshot()` → layout, bounding boxes, computed styles
  - `DOM.getDocument(pierce: true)` → full DOM tree, including shadow DOM
  - `Accessibility.getFullAXTree()` → accessibility tree for all frames
  - `Page.getLayoutMetrics()` → viewport size and device pixel ratio

Running in parallel

- All four CDP calls happen concurrently, not sequentially
- Each gives different information; combined they give the full picture
- Accessibility tree from child frames merged into one array
- Cross-origin iframes handled gracefully (exceptions caught)
- Result: a set of DOM nodes each matched to their accessibility node

Part 5: detecting interactive elements

Layer 1: semantic tags and ARIA roles

Elements that are interactive by HTML definition:

- `<a>` → link
- `<button>` → button
- `<input>`, `<select>`, `<textarea>` → form controls
- Elements with role: button, link, checkbox, radio, combobox, menuitem

These are the easy cases; the tag or role identifies them unambiguously.

Layer 2: JavaScript click handlers

Many modern apps use plain `<div>` or `<span>` elements with click handlers.

- `getEventListeners()` is called on candidate elements
- Elements with click listeners get flagged as interactive
- Applies only on pages with fewer than ~10,000 elements (performance limit)
- Stores backend node IDs of elements with JS listeners
- Handles the large gap between semantic HTML and real-world web apps

Part 6: visibility filtering

Three visibility checks

1. **CSS visibility**: `display: none`, `visibility: hidden`, `opacity: 0` → excluded
2. **Viewport intersection**: bounding box outside current viewport → excluded
3. **Parent iframe visibility**: element inside an off-screen iframe → excluded (checks are applied up through the full iframe hierarchy)

Why filtering matters

- A hidden modal’s buttons should not be clickable
- Off-screen navigation links should wait until the user scrolls or they enter the viewport
- Only elements a human could currently see and click belong in the index
- Filtering keeps the numbered list short and actionable
- The filtered set is rebuilt from scratch after every page change

Take-aways

1. Raw HTML is too large; the DOM service extracts only interactive, visible elements.
2. The DOM snapshot gives layout; the accessibility tree gives semantic roles and names. Both are needed.
3. Interactive-element detection has two layers: semantic HTML tags/roles, then JavaScript click-handler inspection.
4. Three visibility checks filter the candidates: CSS visibility, viewport intersection, and iframe hierarchy.
5. The result is a compact, structured description of the actionable page state, ready to be numbered.

What’s next

- **Lecture 2 (this week):** The indexed selector map and the vision option
- **Reading (required before section):** Week 2 reading covers the full pipeline from DOM to numbered list
- **Section (this week):** Dump the selector map for real pages and match indices to on-screen elements